

Jetson Kamera Servisi — AI Agent Kurulum Kılavuzu

Hedef okuyucu: bir Claude / AI agent · Sürüm 1.0 · jetson-camera-service

Bu kılavuz bir insana değil, bir AI agent'a (Claude) hitap eder. Amaç: sıfırdan bir NVIDIA Jetson Orin Nano üzerinde bu kamera servisini kurup çalışır hale getirmek. Komutları sırayla uygula, her adımda beklenen çıktıyı doğrula. Bir adım beklenen çıktıyı vermezse durma noktalarını (sarı kutular) izle — bazı sorunlar yalnızca fiziksel müdahaleyle (kablo takma vb.) çözülür ve bunları senin yapman mümkün değildir; o durumda insana net talimat ver.

0. Önce oku: Altın kurallar

- Kamera yalnızca NVIDIA Argus üzerinden çalışır. `v4l2` ile ham erişim bu donanımda çalışmaz (sensör ham Bayer verir). Test ederken daima `nvarguscamerasrc` kullan.
- Bu SKU'da donanımsal video encoder (NVENC) YOKTUR. Servis yazılımsal `x264enc` kullanır. `nvv4l2h264enc` arama, yok.
- Argus tek oturumludur. Aynı anda yalnızca tek bir süreç kameraya erişebilir. Test kaydı alırken servis çalışıyorsa çakışır — önce birini durdur.
- Her şey `/mnt/recordings` (NVMe SSD) altındadır: kod, veritabanı, kayıtlar. SD kart yalnızca OS içindir. SSD mount değilse hiçbir şey başlamaz.
- `install.sh` kamera sürücüsünü / device-tree overlay'ini KURMAZ. IMX519 sürücüsü bir ön koşuldur (Bölüm 3 ve 5).
- Port'un tek doğru kaynağı `config/config.yaml`'dir (varsayılan 8080). Kod içine sabit port yazma.
- Kamera "görünmüyor" hatalarının çoğu fiziksel bağlantıdır ve senin çözemeyeceğin türdendir. Bölüm 9'daki teşhisi uygula, gerekirse insana kablo talimatı ver.

1. Sistem nedir

Tarayıcıdan uzaktan yönetilen bir kamera veri toplama servisi. Amaç: görüntü işleme modeli eğitimi için video toplamak. Cihaza fiziksel erişim olmayacağı varsayımıyla tasarlanmış — kayıt başlat/durdur, dosya yönetimi, servis restart, ayarlar hepsi bir web dashboard'undan yapılır. Backend: FastAPI (Python). Frontend: vanilla JS. Veritabanı: SQLite. Kamera: GStreamer + Argus.

2. Donanım ön koşulları (kurulumdan önce doğrula)

Bileşen	Beklenen	Doğrulama komutu
Kart	Jetson Orin Nano Dev Kit	<code>tr -d '\0' < /proc/device-tree/model</code>
OS	JetPack 6.2.1, Ubuntu 22.04, aarch64	<code>uname -m → aarch64 ; cat /etc/os-release</code>
Kamera	Arducam B0371 / Sony IMX519, CSI	Bölüm 3'te
Depolama	NVMe SSD, /mnt/recordings'e mount	<code>mountpoint /mnt/recordings</code>
İnternet	apt + pip için gerekli	<code>curl -sI https://github.com</code>
Kullanıcı	sudo yetkili, root DEĞİL	<code>id</code> (root ile install.sh çalıştırma)

3. KRİTİK: Bu projeye özgü gerçekler

Bunları bilmezsen doğrulanmış ama zor teşhis edilen arızalarla saatler kaybedersin.

3.1 — Kamera sürücüsü install.sh'in dışındadır. IMX519 kernel modülü ve device-tree overlay'i önceden kurulmuş olmalı. Doğrula:

```
lsmod | grep imx519          # imx519 modülü listelenmeli
ls /boot/*.dtbo | grep imx519 # imx519 overlay dosyaları olmalı
```

Eğer yoksa, servis kodu kurulsa bile kamera hiç açılmaz. Arducam'in IMX519 için JetPack sürücü kurulum adımları (kendi kurulum scripti + `/boot/extlinux/extlinux.conf`'ta `OVERLAYS` `.../imx519...dtbo` satırı) önce tamamlanmalı. Bu adım bu depoya dahil değildir.

3.2 — PrivateTmp=false zorunlu. Argus istemcisi `nvargus-daemon`'a `/tmp/argus_socket` üzerinden bağlanır. `systemd` biriminde `PrivateTmp=true` olursa bu soket görünmez olur ve kamera hiç açılmaz — üstelik sağlık kontrolü kamerayı "hazır" raporlayabilir, hata ancak kayıt/önizleme başlarken çıkar. Depodaki `systemd` birimi zaten `PrivateTmp=false`; değiştirme.

3.3 — NVENC yok. Bu Orin Nano SKU'sunda donanım H.264 encoder bulunmaz. Servis `x264enc` (yazılım) kullanır. Doğru: `gst-inspect-1.0 x264enc` çıktı vermelidir; `gst-inspect-1.0 nvv4l2h264enc` boş dönerse bu normaldir.

4. Kodu edinme

Depo `private`'tir (github.com/Mehmet5371/jetson-camera-service). Temiz bir makinede klonlamak için GitHub kimlik doğrulaması gerekir (SSH anahtarı veya PAT). Kod zaten SSD'deyse bu bölümü atla.

```
# SSH anahtarıyla (anahtar GitHub hesabına eklenmiş olmalı):
git clone git@github.com:Mehmet5371/jetson-camera-service.git \
  /mnt/recordings/jetson-camera-service
```

DUR: Temiz makinede genelde SSH anahtarı/kimlik yoktur. Bu adımı sen tek başına tamamlayamazsın — insana "GitHub'da bir SSH anahtarı ekle veya bir PAT ver" diye net talimat ver, sonra devam et. Kimlik bilgisi (parola/token) İSTEME; insanın kendi tarafında yapmasını iste.

5. SSD'yi hazırla (mount değilse)

```
lsblk          # NVMe'yi bul (örn. nvme0n1p1)
sudo mkfs.ext4 /dev/nvme0n1p1 # YALNIZCA boş/yeni SSD ise – veriyi siler!
sudo mkdir -p /mnt/recordings
UUID=$(sudo blkid -s UUID -o value /dev/nvme0n1p1)
echo "UUID=$UUID /mnt/recordings ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab
sudo mount -a
mountpoint /mnt/recordings # 'is a mountpoint' görmelisin
```

mkfs YALNIZCA boş bir SSD içindir. Üzerinde veri olan bir diski asla formatlama. Emin değilsen `lsblk -f` ile dosya sistemi/etiket kontrol et ve insana sor.

6. Kurulum: install.sh

Tek komut. İdempotenttir: mevcut `config.yaml`, `.env`, veritabanı veya kayıtları silmez, yalnızca eksikleri oluşturur.

```
cd /mnt/recordings/jetson-camera-service
./install.sh # sudo yetkili normal kullanıcı; root olarak DEĞİL
```

Sırayla: sistem uyumluluk kontrolü → SSD mount kontrolü → apt paketleri `python3.10-venv`, `ffmpeg`, `v4l-utils`, `i2c-tools`, `gststreamer1.0-tools` → `jetcam` sistem kullanıcısı (video+i2c

gruplarında) → veri izinleri → Python venv + bağımlılıklar → `config.yaml` → `.env` (rastgele SECRET_KEY + admin parolası üretir) → Alembic migration → systemd birimi → dar kapsamlı sudoers → servisi başlat → sağlık kontrolü.

İlk kurulum parolasını yakala. `.env` ilk kez üretildiğinde `install.sh` rastgele bir admin parolasını terminale BİR KEZ yazdırır (Kullanıcı: admin / Parola: ...). Bu satırı çıktıdan yakala ve insana ilet — bir daha gösterilmez. `.env` zaten varsa parola korunur, yeni üretilmez.

7. Doğrulama sırası (kurulumdan sonra, bu sırayla)

7.1 — Servis sağlığı

```
curl -s http://127.0.0.1:8080/api/health
# beklenen: {"success":true,"data":{"status":"ok",...}}
sudo systemctl status jetson-camera-dashboard.service # active (running)
```

7.2 — Kamera donanımı (kamerayı açmadan sorgular)

```
./scripts/diagnose-camera.sh
```

Sağlıklı sistemde: `/dev/video0` mevcut; media topolojisinde `imx519` entity'si NVCSI'ye bağlı; kernel logunda `imx519 ... bound (fail YOK)`; `nvguscamerasrc` ve `x264enc` mevcut.

7.3 — Argus canlı yakalama (kesin test)

```
gst-launch-1.0 nvguscamerasrc sensor-id=0 num-buffers=1 \
! 'video/x-raw(memory:NVMM),width=1920,height=1080' ! fakesink
```

BAŞARILI: GST_ARGUS: Available Sensor modes listelenir, Setup Complete yazar ve Freeing pipeline ile temiz kapanır. Kamera çalışıyor demektir.

BAŞARISIZ: No cameras available → kamera algılanmıyor, Bölüm 9'a git.

7.4 — Uçtan uca kayıt (servisi durdurup)

```
# Argus tek oturumlu: önce servisi durdur, yoksa çakışır
sudo systemctl stop jetson-camera-dashboard.service
./scripts/test-recording.sh 10 # 10 sn gerçek kayıt + ffprobe doğrulama
sudo systemctl start jetson-camera-dashboard.service
```

8. Kamera görünmüyorsa: hızlı teşhis

Üç bağımsız sinyali kontrol et:

```
ls /dev/video0 # yoksa: sensör enumerate olmamış
sudo journalctl -k -b | grep imx519 # probe sonucu
i2cdetect -y -r 10 ; i2cdetect -y -r 9 # sensör 0x1a görünüyor mu
```

9. Hata: imx519 ... error during i2c read probe (-121)

-121 = EREMOTEIO. Kernel sensörü doğru adreste `0x1a` çağırıyor ama sensör I2C'de hiç ACK vermiyor. `i2cdetect`'te ilgili bus (9/10) tamamen boşsa (EEPROM bile yok), modüle elektriksel olarak hiç ulaşılamıyordur. Bu bir yazılım hatası DEĞİL, fiziksel bağlantı hatasıdır.

DUR — bunu sen çözemezsin. İnsana şu talimatı ver: 1) `sudo poweroff` (kabloyu enerji varken takip çıkarma).

2) CSI şerit kablосunu her iki uçta (Jetson + kamera modülü) aç; metal kontak yüzü doğru yöne bakacak şekilde düz ve sonuna kadar it, mandalı kapat.

3) En sık hata ters takmadır — hata birebir tekrarlıyorsa kabloyu ters çevirip dene.

4) Hâlâ -121 ise: kabloyu değiştir; o da olmazsa modül/port arızalı olabilir.

Her denemeden sonra Bölüm 8'deki üç kontrolü tekrar çalıştır. Kablo taraması boş kaldıkça yazılımla yapılabilecek bir şey yoktur.

10. Servis yönetimi

Amaç	Komut
Başlat / durdur / durum (kurulumdan sonra)	<code>sudo systemctl start stop status jetson-camera-dashboard.service</code>
Loglar	<code>sudo journalctl -u jetson-camera-dashboard.service -f</code>
systemd YOKKEN elle başlat	<code>./scripts/start-dashboard.sh --background</code>
Elle başlatılanı durdur	<code>pkill -INT -f 'uvicorn app.main:app'</code>
Açılıştan oto-başlat + kiosk	<code>./scripts/setup-autostart.sh</code>

Argus'ta takılan süreç: Nadiren durdurulan süreç Argus I/O'sunda (kesintisiz uyku, D state) asılı kalır; SIGKILL bile teslim edilemez. Çözüm: `sudo systemctl restart nvargus-daemon` — takılı istemcileri serbest bırakır.

11. İlk giriş

- Adres: `http://<jetson-ip>:8080` (yerel), veya cihazda `http://127.0.0.1:8080`.
- Kullanıcı: `admin`. Parola: `install.sh` ilk kurulumda ürettiği rastgele parola (Bölüm 6). İlk girişte değişim zorunludur.
- Parola veritabanında Argon2 hash olarak saklanır; düz metin hiçbir yerde yoktur. Unutulursa `backend/app/security/passwords.py`'deki `hash_password()` ile sıfırlanır (users tablosu, `must_change_password=True` bırak).

12. Ne zaman dur ve insana danış

- GitHub kimlik doğrulaması gerektiğinde (private repo klonlama) — Bölüm 4.
- SSD formatlama kararı — üzerinde veri olup olmadığından emin değilsen — Bölüm 5.
- Kamera -121 / fiziksel bağlantı — kablo takma senin yapamayacağın iştir — Bölüm 9.
- IMX519 sürücüsü/overlay eksikse — Arducam JetPack kurulumu ayrı bir süreçtir — Bölüm 3.1.
- Üretilen admin parolası — çıktıdan yakalayıp insana ilet, bir daha gösterilmez.

Bu kılavuz jetson-camera-service deposundaki `install.sh`, `scripts/` ve `docs/` içeriğinden türetilmiştir. Komutlar değişirse depo kaynak kabul edilir. Ayrıntı: `docs/INSTALLATION.md`, `docs/CAMERA_SETUP.md`, `docs/TROUBLESHOOTING.md`, `docs/ARCHITECTURE.md`.